

Embedded systems

Instruction Set Overview

This is a complete overview of the instruction set for the Motorola MC9S12DT256 microprocessor. Some of the groups are irrelevant if you are just starting to learn about programming the processor. The basic important groups are

- Load and store instructions
- Transfer and exchange instructions
- Move instructions
- Addition and subtraction instructions
- Decrement and increment instructions
- Compare and test instructions
- Boolean logic instructions
- Clear, complement and negate instructions
- Multiplication and division instructions
- Bit test and manipulation instructions
- Shift and rotate instructions
- Short branch instructions
- Long branch instructions
- Bit condition branch instructions
- Loop primitive instructions
- Jump and subroutine instructions

The next group to study would be

- Maximum and minimum instructions
- Multiply and accumulate instruction
- Interrupt instructions
- Index manipulation instructions
- Stacking instructions
- Condition code instructions



and the last group would be

- Binary-coded decimal instructions
- Fuzzy logic instructions
- Table interpolation instructions
- Pointer and index calculation instructions
- Stop and wait instructions
- Background mode and null instructions

Load and store instructions

Mnemonic	Function	Operation
Load instructions		
LDAA	Load A	$(M) \rightarrow A$
LDAB	Load B	$(M) \rightarrow B$
LDD	Load D	$(M:M+1) \rightarrow A:B$
LDS	Load SP	$(M:M+1) \rightarrow SP_H:SP_L$
LDX	Load index register X	$(M:M+1) \rightarrow X_H:X_L$
LDY	Load index register Y	$(M:M+1) \rightarrow Y_H:Y_L$
LEAS	Load effective address into SP	Effective address $\rightarrow SP$
LEAX	Load effective address into X	Effective address $\rightarrow X$
LEAY	Load effective address into Y	Effective address $\rightarrow Y$
Store instructions		
STAA	Store A	$(A) \rightarrow M$
STAB	Store B	$(B) \rightarrow M$
STD	Store D	$(A) \rightarrow M, (B) \rightarrow M+1$
STS	Store SP	$(SP_H:SP_L) \rightarrow M:M+1$
STX	Store X	$(X_H:X_L) \rightarrow M:M+1$
STY	Store Y	$(Y_H:Y_L) \rightarrow M:M+1$

Transfer and exchange instructions

Mnemonic	Function	Operation
Transfer instructions		
TAB	Transfer A to B	$(A) \rightarrow B$
TAP	Transfer A to CCR	$(A) \rightarrow CCR$
TBA	Transfer B to A	$(B) \rightarrow A$
TFR	Transfer register to register	$(A, B, CCR, D, X, Y \text{ or } SP) \rightarrow A, B, CCR, D, X, Y \text{ or } SP$
TPA	Transfer CCR to A	$(CCR) \rightarrow A$
TSX	Transfer SP to X	$(SP) \rightarrow X$
TSY	Transfer SP to Y	$(SP) \rightarrow Y$

Transfer and exchange instructions cont.

Mnemonic	Function	Operation
TXS	Transfer X to SP	$(X) \rightarrow SP$
TYS	Transfer Y to SP	$(Y) \rightarrow SP$
Exchange instructions		
EXG	Exchange register to register	$(A, B, CCR, D, X, Y \text{ or } SP) \leftrightarrow A, B, CCR, D, X, Y \text{ or } SP$
XGDX	Exchange D with X	$(D) \leftrightarrow (X)$
XGDY	Exchange D with Y	$(D) \leftrightarrow (Y)$
Sign extension instruction		
SEX	Store SP	Sign-extended (A, B or CCR) $\rightarrow$ D, X, Y or SP

Move instructions

Mnemonic	Function	Operation
MOVB	Move byte (8-bit)	$(M_1) \rightarrow M_2$
MOVW	Move word (16-bit)	$(M_1:M+1_1) \rightarrow M_2:M+1_2$

Addition and subtraction instructions

Mnemonic	Function	Operation
Addition instructions		
ABA	Add B to A	$(A) + (B) \rightarrow A$
ABX	Add B to X	$(B) + (X) \rightarrow X$
ABY	Add B to Y	$(B) + (Y) \rightarrow Y$
ADCA	Add with carry to A	$(A) + (M) + C \rightarrow A$
ADCB	Add with carry to B	$(B) + (M) + C \rightarrow B$
ADDA	Add without carry to A	$(A) + (M) \rightarrow A$
ADDB	Add without carry to B	$(B) + (M) \rightarrow B$
ADDD	Add to D	$(A:B) + (M:M+1) \rightarrow D$
Subtraction instructions		
SBA	Subtract B from A	$(A) - (B) \rightarrow A$
SBCA	Subtract with borrow from A	$(A) - (M) - C \rightarrow A$
SBCB	Subtract with borrow from B	$(B) - (M) - C \rightarrow B$
SUBA	Subtract memory from A	$(A) - (M) \rightarrow A$
SUBB	Subtract memory from B	$(B) - (M) \rightarrow B$
SUBD	Subtract memory from D (A:B)	$(D) + (M:M+1) \rightarrow D$

Binar-Coded Decimal (BCD) instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
ABA	Add B to A	$(A) + (B) \rightarrow A$
ADCA	Add with carry to A	$(A) + (M) + C \rightarrow A$
ADCB ⁽¹⁾	Add with carry to B	$(B) + (M) + C \rightarrow B$
ADDA ⁽¹⁾	Add memory to A	$(A) + (M) \rightarrow A$
ADDB	Add memory to B	$(B) + (M) \rightarrow B$
DAA	Decimal adjust A	$(A)_{10}$

1. These instructions are not normally used for BCD operations because although they affect H correctly. They do not leave the result in the correct accumulator (A) to be used with the DAA instruction. Thus additional steps would be needed to adjust the result to correct BCD form

Decrement and increment instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
<i>Decrement instructions</i>		
DEC	Decrement memory	$(M) - \$01 \rightarrow M$
DECA	Decrement A	$(A) - \$01 \rightarrow A$
DECB	Decrement B	$(B) - \$01 \rightarrow B$
DES	Decrement SP	$(SP) - \$0001 \rightarrow SP$
DEX	Decrement X	$(X) - \$0001 \rightarrow X$
DEY	Decrement Y	$(Y) - \$0001 \rightarrow Y$
<i>Increment instructions</i>		
INC	Increment memory	$(M) + \$01 \rightarrow M$
INCA	Increment A	$(A) + \$01 \rightarrow A$
INCB	Increment B	$(B) + \$01 \rightarrow B$
INS	Increment SP	$(SP) + \$0001 \rightarrow SP$
INX	Increment X	$(X) + \$0001 \rightarrow X$
INY	Increment Y	$(Y) + \$0001 \rightarrow Y$

Compare and test instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
<i>Compare instructions</i>		
CBA	Compare A to B	$(A) - (B)$
CMPA	Compare A to memory	$(A) - (M)$

Compare and test instructions cont.

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
CMPB	Compare B to memory	$(B) - (M)$
CPD	Compare D to memory (16-bit)	$(A:B) - (M:M+1)$
CPS	Compare SP to memory (16-bit)	$(SP) - (M:M+1)$
CPX	Compare X to memory (16-bit)	$(X) - (M:M+1)$
CPY	Compare Y to memory (16-bit)	$(Y) - (M:M+1)$
<i>Test instructions</i>		
TST	Test memory for zero or minus	$(M) - \$00$
TSTA	Test A for zero and minus	$(A) - \$00$
TSTB	Test B for zero and minus	$(B) - \$00$

Boolean logic instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
ANDA	AND A with memory	$(A) \cdot (M) \rightarrow A$
ANDB	AND B with memory	$(A) \cdot (M) \rightarrow B$
ANDCC	AND CCR with memory (clear CCR bits)	$(CCR) \cdot (M) \rightarrow CCR$
EORA	Exclusive OR A with memory	$(A) * (M) \rightarrow A$
EORB	Exclusive OR B with memory	$(B) * (M) \rightarrow B$
ORAA	OR A with memory	$(A) + (M) \rightarrow A$
ORAB	OR B with memory	$(B) + (M) \rightarrow B$
ORCC	OR CCR with memory (set CCR bits)	$(CCR) + (M) \rightarrow CCR$

Clear, complement and negate instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
CLC	Clear C bit in CCR	$0 \rightarrow C$
CLI	Clear I bit in CCR	$0 \rightarrow I$
CLR	Clear memory	$\$00 \rightarrow M$
CLRA	Clear A	$\$00 \rightarrow A$
CLRB	Clear B	$\$00 \rightarrow B$
CLV	Clear V bit in CCR	$0 \rightarrow V$
COM	1's complement memory	$\$FF - (M) \rightarrow M$ or $(/M) \rightarrow M$
COMA	1's complement A	$\$FF - (A) \rightarrow A$ or $(/A) \rightarrow A$
COMB	1's complement B	$\$FF - (B) \rightarrow B$ or $(/B) \rightarrow B$
NEG	2's complement memory	$\$00 - (M) \rightarrow M$ or $(/M) + 1 \rightarrow M$
NEGA	2's complement A	$\$00 - (A) \rightarrow A$ or $(/A) + 1 \rightarrow A$

Clear, complement and negate instructions cont.

Mnemonic	Function	Operation
NEGB	2's complement B	$\$00 - (B) \rightarrow B$ or $(/B) + 1 \rightarrow B$

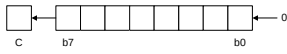
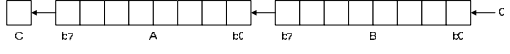
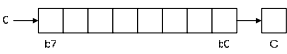
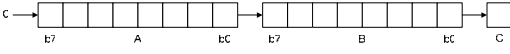
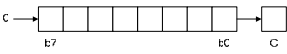
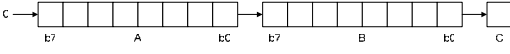
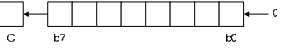
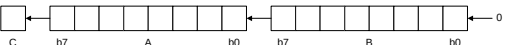
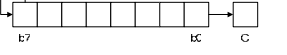
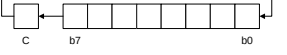
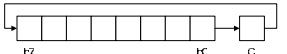
Multiplication and division instructions

Mnemonic	Function	Operation
Multiplication instructions		
EMUL	16 by 16 multiply (unsigned)	$(D) \times (Y) \rightarrow Y:D$
EMULS	16 by 16 multiply (signed)	$(D) \times (Y) \rightarrow Y:D$
MUL	8 by 8 multiply (unsigned)	$(A) \times (B) \rightarrow A:B$
Division instructions		
EDIV	32 by 16 divide (unsigned)	$(Y:D) \div (X) \rightarrow Y$ Remainder $\rightarrow D$
EDIVS	32 by 16 divide (signed)	$(Y:D) \div (X) \rightarrow Y$ Remainder $\rightarrow D$
FDIV	16 by 16 fractional divide	$(D) \div (X) \rightarrow X$ Remainder $\rightarrow D$
IDIV	16 by 16 integer divide (unsigned)	$(D) \div (X) \rightarrow X$ Remainder $\rightarrow D$
IDIVS	16 by 16 integer divide (signed)	$(D) \div (X) \rightarrow X$ Remainder $\rightarrow D$

Bit test and manipulation instructions

Mnemonic	Function	Operation
BCLR	Clear bits in memory	$(M) \cdot (/mm) \rightarrow M$
BITA	Bit test A	$(A) \cdot (M)$
BITB	Bit test B	$(B) \cdot (M)$
BSET	Set bits in memory	$(M) + (mm) \rightarrow M$

Shift and rotate instructions

Mnemonic	Function	Operation
Logical shifts		
LSL LSLA LSLB	Logic shift left memory Logic shift left A Logic shift left B	
LSLD	Logic shift left D	
LSR LSRA LSRB	Logic shift right memory Logic shift right A Logic shift right B	
LSRD	Logic shift right D	
LSR LSRA LSRB	Logic shift right memory Logic shift right A Logic shift right B	
LSRD	Logic shift right D	
Arithmetic Shifts		
ASL ASLA ASLB	Arithmetic shift left memory Arithmetic shift left A Arithmetic shift left B	
ASLD	Arithmetic shift left D	
ASR ARA ARB	Arithmetic shift right memory Arithmetic shift right A Arithmetic shift right B	
Rotates		
ROL ROLA ROLB	Rotate left memory through carry Rotate left A through carry Rotate left B through carry	
ROR RORA RORB	Rotate right memory through carry Rotate right A through carry Rotate right B through carry	

Fuzzy logic instructions

Mnemonic	Function	Operation
MEM	Membership function	$\mu(\text{grade}) \rightarrow M_{(Y)}$ $(X) + 4 \rightarrow X; (Y) + 1 \rightarrow A; A$ unchanged. If $(A) < P1$ or $(A) > P2$ then $\mu = 0$, else $\mu = \text{MIN}[(A - P1) \times S1, (P2 - A) \times S2, \$FF]$ where A = current crisp input value X points to a 4-byte data structure that describes a trapezoidal membership function as base intercept points and slopes $(P1, P2, S1, S2)$. Y points at fuzzy input (RAM location)
REV	MIN – MAX rule avaluation	Find smallest rule input (MIN). Store to rule outputs unless fuzzy output is larger (MAX). Rules are unweighted. Each rule input is an 8-bit offset from a base address in Y . Each rule output is an 8-bit offset from a base address in Y . $\$FE$ separates rule inputs from rule outputs. $\$FF$ terminates the rule list. REV can be interrupted
REVV	MIN – MAX rule avaluation	Find smallest rule input (MIN). Multiply by a rule weighting factor (optional). Store to rule outputs unless fuzzy output is larger (MAX). Each rule input is an 16-bit address of a fuzzy input. Each rule output is an 16-bit address of a fuzzy input. Address $\$FFE$ separates rule inputs from rule outputs. $\$FFF$ terminates the rule list. Weights are 8-bit values in a separate table. REVW can be interrupted
WAV	Calculates numerator (sum of products) and denominator (sum of weights) for weighted average calculation. Results are placed in correct registers for EDIV immediately after WAV	$\sum_{i=1}^B S_i \cdot F_i \rightarrow Y : D$ $\sum_{i=1}^B F_i \rightarrow X$
WAVR	Resumes execution of interrupted WAV instruction	Recover immediate results from stack rather than initializing them to 0.

Minimum and maximum instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
<i>Minimum instructions</i>		
EMIND	MIN of two unsigned 16-bit values, result to accumulator	$\text{MIN}[(D), (M:M+1)] \rightarrow D$
EMINM	MIN of two unsigned 16-bit values, result to memory	$\text{MIN}[(D), (M:M+1)] \rightarrow M:M+1$
MINA	MIN of two unsigned 8-bit values, result to accumulator	$\text{MIN}[(A), (M)] \rightarrow A$
MINM	MIN of two unsigned 8-bit values, result to memory	$\text{MIN}[(A), (M)] \rightarrow M$
<i>Maximum instructions</i>		
EMAXD	MAX of two unsigned 16-bit values, result to accumulator	$\text{MAX}[(D), (M:M+1)] \rightarrow D$
EMAXM	MAX of two unsigned 16-bit values, result to memory	$\text{MAX}[(D), (M:M+1)] \rightarrow M:M+1$
MAXA	MAX of two unsigned 8-bit values, result to accumulator	$\text{MAX}[(A), (M)] \rightarrow A$
MAXM	MAX of two unsigned 8-bit values, result to memory	$\text{MAX}[(A), (M)] \rightarrow M$

Multiply and accumulate instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
EMACS	Multiply and accumulate (signed) 16 bit by 16 bit $\rightarrow$ 32 bit	$[(M_{(X)}:M_{(X+1)}) \times (M_{(Y)}:M_{(Y+1)})] + (M \rightarrow M+3) \rightarrow M \rightarrow M+3$

Table interpolation instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
ETBL	16-bit table lookup and interpolate (no indirect addressing modes allowed)	$(M:M+1) + [(B) \times ((M+2:M+3) - (M:M+1))] \rightarrow D$ Initialize B and index before ETBL. <ea> points to the first table entry (M:M+1). B is fractional part of lookup value
TBL	8-bit table lookup and interpolate (no indirect addressing modes allowed)	$(M) + [(B) \times ((M+1) - (M))] \rightarrow A$ Initialize B and index before TBL. <ea> points to the first table entry (M:M+1). B is fractional part of lookup value

Short branch instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Equation or operation</i>	
<i>Unary branches</i>			
BRA	Branch always	$1 = 1$	
BRN	Branch never	$1 = 0$	
<i>Simple branches</i>			
BCC	Branch if carry clear	$C = 0$	
BCS	Branch if carry set	$C = 1$	
BEQ	Branch if equal	$Z = 1$	
BMI	Branch if minus	$N = 1$	
BNE	Branch if not equal	$Z = 0$	
BPL	Branch if plus	$N = 0$	
BVC	Branch if overflow clear	$V = 0$	
BVS	Branch if overflow set	$V = 1$	
<i>Unsigned branches</i>			
		<i>Relation</i>	
BHI	Branch if higher	$R > M$	$C + Z = 0$
BHS	Branch if higher or same	$R \geq M$	$C = 0$
BLO	Branch if lower	$R < M$	$C = 1$
BLS	Branch if lower or same	$R \leq M$	$C + Z = 1$
<i>Signed branches</i>			
BGE	Branch if greater than or equal	$R \geq M$	$N * V = 0$
BGT	Branch if greater than	$R > M$	$Z + (N * V) = 0$
BLE	Branch if less than or equal	$R \leq M$	$Z + (N * V) = 1$
BLT	Branch if less than	$R < M$	$N * V = 1$

Long branch instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Equation or operation</i>
<i>Unary branches</i>		
LBRA	Long branch always	1 = 1
LBRN	Long branch never	1 = 0
<i>Simple branches</i>		
LBCC	Long branch if carry clear	C = 0
LBCS	Long branch if carry set	C = 1
LBEQ	Long branch if equal	Z = 1
LBMI	Long branch if minus	N = 1
LBNE	Long branch if not equal	Z = 0
LBPL	Long branch if plus	N = 0
LBVC	Long branch if overflow clear	V = 0
LBVS	Long branch if overflow set	V = 1

Long branch instructions cont.

<i>Mnemonic</i>	<i>Function</i>	<i>Equation or operation</i>
<i>Unsigned branches</i>		
LBHI	Long branch if higher	$C + Z = 0$
LBHS	Long branch if higher or same	$C = 0$
LBLO	Long branch if lower	$C = 1$
LBLS	Long branch if lower or same	$C + Z = 1$
<i>Signed branches</i>		
LBGE	Long branch if greater than or equal	$N * V = 0$
LBGT	Long branch if greater than	$Z + (N * V) = 0$
LBLE	Long branch if less than or equal	$Z + (N * V) = 1$
LBLT	Long branch if less than	$N * V = 1$

Bit condition branch instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Equation or operation</i>
BRCLR	Branch if selected bits clear	$(M) \cdot (mm) \rightarrow 0$
BRSET	Branch if selected bits set	$(/M) \cdot (mm) \rightarrow 0$

Loop primitive instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Equation or operation</i>
DBEQ	Decrement counter and branch if = 0 (counter = A, B, D, X, Y or SP)	(counter) - 1 → counter if (counter) = 0 then branch else continue to next instruction
DBNE	Decrement counter and branch if ≠ 0 (counter = A, B, D, X, Y or SP)	(counter) - 1 → counter if (counter) not = 0 then branch else continue to next instruction
IBEQ	Increment counter and branch if = 0 (counter = A, B, D, X, Y or SP)	(counter) + 1 → counter if (counter) = 0 then branch else continue to next instruction
IBNE	Increment counter and branch if ≠ 0 (counter = A, B, D, X, Y or SP)	(counter) + 1 → counter if (counter) not = 0 then branch else continue to next instruction
TBEQ	Test counter and branch if = 0 (counter = A, B, D, X, Y or SP)	If (counter) = 0 then branch else continue to next instruction
TBNE	Test counter and branch if ≠ 0 (counter = A, B, D, X, Y or SP)	If (counter) not = 0 then branch else continue to next instruction

Jump and subroutine instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
BSR	Branch to subroutine	$SP - 2 \rightarrow SP$ $RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$
CAL	Call subroutine in expanded memory	$SP - 2 \rightarrow SP$ $RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP$ $(PPAGE) \rightarrow M_{(SP)}$ $Page \rightarrow PPAGE$ $Subroutine\ address \rightarrow PC$
JMP	Jump	$Address \rightarrow PC$
JSR	Jump to subroutine	$SP - 2 \rightarrow SP$ $RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $Subroutine\ address \rightarrow PC$
RTC	Return from call	$M_{(SP)} \rightarrow PPAGE$ $SP + 1 \rightarrow SP$ $M_{(SP)}:M_{(SP+1)} \rightarrow PC_H:PC_L$ $SP + 2 \rightarrow SP$
RTS	Return from subroutine	$M_{(SP)}:M_{(SP+1)} \rightarrow PC_H:PC_L$ $SP + 2 \rightarrow SP$

Interrupt instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
RTI	Return from interrupt	$(M_{(SP)}) \rightarrow CCR; (SP) + \$0001 \rightarrow SP$ $(M_{(SP)}:M_{(SP+1)}) \rightarrow B; (SP) + \$0002 \rightarrow SP$ $(M_{(SP)}:M_{(SP+1)}) \rightarrow X_H:X_L; (SP) + \$0004 \rightarrow SP$ $(M_{(SP)}:M_{(SP+1)}) \rightarrow PC:PC_L; (SP) + \$0002 \rightarrow SP$ $(M_{(SP)}:M_{(SP+1)}) \rightarrow Y_H:Y_L; (SP) + \$0004 \rightarrow SP$
SWI	Software interrupt	$SP - 2 \rightarrow SP; RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; Y_H:Y_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; X_H:X_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; B:A \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 1 \rightarrow SP; CCR \rightarrow M_{(SP)}$
TRAP	Unimplemented opcode interrupt	$SP - 2 \rightarrow SP; RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; Y_H:Y_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; X_H:X_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; B:A \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 1 \rightarrow SP; CCR \rightarrow M_{(SP)}$

Index manipulation instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
<i>Addition instructions</i>		
ABX	Add B to X	$(B) + (X) \rightarrow X$
ABY	Add B to Y	$(B) + (Y) \rightarrow Y$
<i>Compare instructions</i>		
CPS	Compare SP to memory	$(SP) - (M:M+1)$
CPX	Compare X to memory	$(X) - (M:M+1)$
CPY	Compare Y to memory	$(Y) - (M:M+1)$
<i>Load instructions</i>		
LDS	Load SP from memory	$(M:M+1) \rightarrow SP$
LDX	Load X from memory	$(M:M+1) \rightarrow X$
LDY	Load Y from memory	$(M:M+1) \rightarrow Y$
LEAS	Load effective address into SP	Effective address $\rightarrow SP$
LEAX	Load effective address into X	Effective address $\rightarrow X$
LEAY	Load effective address into Y	Effective address $\rightarrow Y$
<i>Store instructions</i>		
STS	Store SP in memory	$(SP) \rightarrow M:M+1$
STX	Store X in memory	$(X) \rightarrow M:M+1$
STY	Store Y in memory	$(Y) \rightarrow M:M+1$
<i>Transfer instructions</i>		
TFR	Transfer register to register	$(A, B, CCR, D, X, Y \text{ or } SP) \rightarrow A, B, CCR, D, X, Y \text{ or } SP$
TSX	Transfer SP to X	$(SP) \rightarrow X$
TSY	Transfer SP to Y	$(SP) \rightarrow Y$
TXS	Transfer X to SP	$(X) \rightarrow SP$
TYS	Transfer Y to SP	$(Y) \rightarrow SP$
<i>Exchange instructions</i>		
EXG	Exchange register to register	$(A, B, CCR, D, X, Y \text{ or } SP) \leftrightarrow A, B, CCR, D, X, Y \text{ or } SP$
XGDX	Exchange D with X	$(D) \leftrightarrow (X)$
XGDY	Exchange D with Y	$(D) \leftrightarrow (Y)$

Stacking instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
<i>Stack pointer instructions</i>		
CPS	Compare SP to memory	$(SP) - (M:M+1)$
DES	Decrement SP	$(SP) - 1 \rightarrow SP$
INS	Increment SP	$(SP) + 1 \rightarrow SP$
LDS	Load SP	$(M:M+1) \rightarrow SP$
LEAS	Load effective address into SP	Effective address $\rightarrow SP$
STS	Store SP	$(SP) \rightarrow M:M+1$

Stacking instructions cont.

Mnemonic	Function	Operation
TSX	Transfer SP to X	$(SP) \rightarrow X$
TSY	Transfer SP to Y	$(SP) \rightarrow Y$
TXS	Transfer X to SP	$(X) \rightarrow SP$
TYS	Transfer Y to SP	$(Y) \rightarrow SP$
Stack operation instructions		
PSHA	Push A	$(SP) - 1 \rightarrow SP; (A) \rightarrow M_{(SP)}$
PSHB	Push B	$(SP) - 1 \rightarrow SP; (B) \rightarrow M_{(SP)}$
PSHC	Push CCR	$(SP) - 1 \rightarrow SP; (CCR) \rightarrow M_{(SP)}$
PSHD	Push D	$(SP) - 2 \rightarrow SP; (A:B) \rightarrow M_{(SP)}:M_{(SP+1)}$
PSHX	Push X	$(SP) - 2 \rightarrow SP; (X) \rightarrow M_{(SP)}:M_{(SP+1)}$
PSHY	Push Y	$(SP) - 2 \rightarrow SP; (Y) \rightarrow M_{(SP)}:M_{(SP+1)}$
PULA	Pull A	$(M_{(SP)}) \rightarrow A; (SP) + 1 \rightarrow SP$
PULB	Pull B	$(M_{(SP)}) \rightarrow B; (SP) + 1 \rightarrow SP$
PULC	Pull CCR	$(M_{(SP)}) \rightarrow CCR; (SP) + 1 \rightarrow SP$
PULD	Pull D	$(M_{(SP)}:M_{(SP+1)}) \rightarrow A:B; (SP) + 2 \rightarrow SP$
PULX	Pull X	$(M_{(SP)}:M_{(SP+1)}) \rightarrow X; (SP) + 2 \rightarrow SP$
PULY	Pull Y	$(M_{(SP)}:M_{(SP+1)}) \rightarrow Y; (SP) + 2 \rightarrow SP$

Pointer and index calculation instructions

Mnemonic	Function	Operation
LEAS	Load result of indexed addressing mode effective address calculation into stack pointer	$r \pm \text{constant} \rightarrow SP$ or $(r) + (\text{accumulator}) \rightarrow SP$ $R = X, Y, SP \text{ or } PC$
LEAX	Load result of indexed addressing mode effective address calculation into X index register	$r \pm \text{constant} \rightarrow X$ or $(r) + (\text{accumulator}) \rightarrow X$ $R = X, Y, SP \text{ or } PC$
LEAY	Load result of indexed addressing mode effective address calculation into Y index register	$r \pm \text{constant} \rightarrow Y$ or $(r) + (\text{accumulator}) \rightarrow Y$ $R = X, Y, SP \text{ or } PC$

Condition code instructions

Mnemonic	Function	Operation
ANDCC	Logical AND CCR with memory	$(CCR) \cdot (M) \rightarrow CCR$
CLC	Clear C bit	$0 \rightarrow C$
CLI	Clear I bit	$0 \rightarrow I$

Condition code instructions cont.

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
CLV	Clear V bit	$0 \rightarrow V$
ORCC	Logical OR CCR with memory	$(CCR) + (M) \rightarrow CCR$
PSHC	Push CCR onto stack	$(M:M+1) \rightarrow SP$
PULC	Pull CCR from stack	$(SP) - 1 \rightarrow SP; (CCR) \rightarrow M_{(SP)}$
SEC	Set C bit	$1 \rightarrow C$
SEI	Set I bit	$1 \rightarrow I$
SEV	Set V bit	$1 \rightarrow V$
TAP	Transfer A to CCR	$(A) \rightarrow CCR$
TPA	Transfer CCR to A	$(CCR) \rightarrow A$

Stop and wait instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
STOP	Stop	$SP - 2 \rightarrow SP; RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; Y_H:Y_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; X_H:X_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; B:A \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 1 \rightarrow SP; CCR \rightarrow M_{(SP)}$ Stop CPU clocks
WAI	Wait for interrupt	$SP - 2 \rightarrow SP; RTN_H:RTN_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; Y_H:Y_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; X_H:X_L \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 2 \rightarrow SP; B:A \rightarrow M_{(SP)}:M_{(SP+1)}$ $SP - 1 \rightarrow SP; CCR \rightarrow M_{(SP)}$

Background mode and null operation instructions

<i>Mnemonic</i>	<i>Function</i>	<i>Operation</i>
BGND	Enter background debug mode	If BDM enabled, enter BDM, else resume normal processing
BRN	Branch never	Does not branch
LBRN	Long branch never	Does not branch
NOP	Null operation	-